

AI Developer Tools Sentiment Intelligence Platform (Final Synced Version)

Workflow

- 1 Load data
- 2 Clean text
- 3 Sentiment analysis
- 4 Feature engineering
- 5 EDA
- 6 Product analytics
- 7 Insights

Notebook designed to run top→bottom without errors.

Imports

```
In [1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import re
import os

from vaderSentiment.vaderSentiment import SentimentIntensityAnalyzer
from sklearn.feature_extraction.text import CountVectorizer

sns.set_style("whitegrid")
```

Load Dataset

```
In [2]: file="ai_tools_sentiment_dataset_800.csv"

if os.path.exists(file):

    df=pd.read_csv(file)

    print("Loaded:",df.shape)

else:

    print("Dataset missing")

df.head()
```

Loaded: (800, 6)

Out[2]:

	tool	text	source	date	synthetic_sentiment_label	feature_tag
0	ChatGPT	ChatGPT inaccurate suggestions sometimes espec...	Tech Blog	2024-07-05	negative	prompts
1	ChatGPT	ChatGPT good documentation support especially ...	G2	2024-07-04	positive	general coding
2	ChatGPT	ChatGPT easy to use especially for debugging.	User Review	2025-05-26	positive	debugging
3	ChatGPT	ChatGPT very helpful assistant especially for ...	Developer Forum	2025-06-08	positive	general coding
4	ChatGPT	ChatGPT improves efficiency especially for pro...	G2	2024-02-14	positive	prompts

Data Cleaning

In [3]:

```
def clean(text):  
  
    text=str(text).lower()  
  
    text=re.sub(r"http\S+", "", text)  
  
    text=re.sub(r"^[a-z ]", "", text)  
  
    text=re.sub(r"\s+", " ", text)  
  
    return text.strip()  
  
df["clean"]=df["text"].apply(clean)  
  
df["date"]=pd.to_datetime(df["date"])
```

Sentiment Analysis (Must run before EDA)

In [4]:

```
analyzer=SentimentIntensityAnalyzer()  
  
df["sentiment_score"]=df["clean"].apply(  
  
    lambda x: analyzer.polarity_scores(x)["compound"]  
  
)  
  
df["sentiment"]=df["sentiment_score"].apply(  
  
    lambda x:"positive" if x>.05  
    else "negative" if x<-.05  
    else "neutral"  
  
)  
  
df.head()
```

Out[4]:

	tool	text	source	date	synthetic_sentiment_label	feature_tag	clean	sentiment
0	ChatGPT	ChatGPT inaccurate suggestions sometimes espec...	Tech Blog	2024-07-05	negative	prompts	chatgpt inaccurate suggestions sometimes espec...	
1	ChatGPT	ChatGPT good documentation support especially ...	G2	2024-07-04	positive	general coding	chatgpt good documentation support especially ...	
2	ChatGPT	ChatGPT easy to use especially for debugging.	User Review	2025-05-26	positive	debugging	chatgpt easy to use especially for debugging	
3	ChatGPT	ChatGPT very helpful assistant especially for ...	Developer Forum	2025-06-08	positive	general coding	chatgpt very helpful assistant especially for ...	
4	ChatGPT	ChatGPT improves efficiency especially for pro...	G2	2024-02-14	positive	prompts	chatgpt improves efficiency especially for pro...	

Feature Engineering

In [5]:

```
FEATURES={  
  
    "speed":["fast","performance"],  
  
    "accuracy":["error","wrong","hallucination"],  
  
    "integration":["integration","aws"],  
  
    "usability":["easy","complex"],  
  
    "productivity":["productivity","faster"]  
}
```

In [6]:

```
def detect_features(text):  
  
    f=[]  
  
    for feature,words in FEATURES.items():  
  
        for word in words:  
  
            if word in text:  
  
                f.append(feature)  
  
                break
```

```
return f
```

```
df["features"]=df["clean"].apply(detect_features)
```

```
In [7]: rows=[]

for _,row in df.iterrows():

    for f in row["features"]:

        rows.append({

            "tool":row["tool"],

            "feature":f,

            "sentiment":row["sentiment_score"]

        })

feature_df=pd.DataFrame(rows)

feature_df.head()
```

```
Out[7]:
```

	tool	feature	sentiment
0	ChatGPT	usability	0.4404
1	ChatGPT	speed	0.3818
2	ChatGPT	productivity	0.3818
3	ChatGPT	usability	0.4404
4	ChatGPT	integration	0.4404

EDA – Dataset Diagnostics

```
In [8]: print("Shape:",df.shape)

print(df.isna().sum())

print(df["tool"].value_counts())

print(df["sentiment"].value_counts())
```

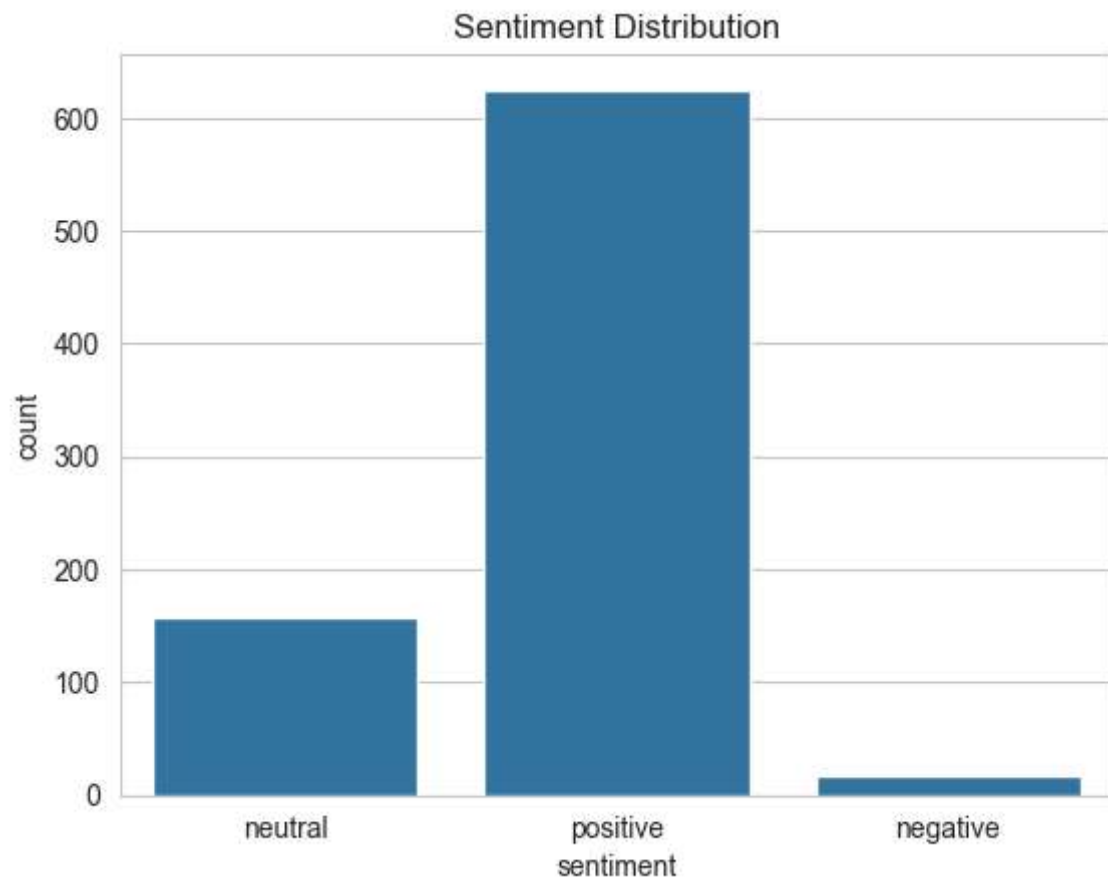
```
Shape: (800, 10)
tool                                0
text                                0
source                              0
date                                0
synthetic_sentiment_label          0
feature_tag                         0
clean                               0
sentiment_score                    0
sentiment                          0
features                           0
dtype: int64
tool
ChatGPT                200
GitHub Copilot         200
Cursor                 200
Amazon Q               200
Name: count, dtype: int64
sentiment
positive    625
neutral     157
negative     18
Name: count, dtype: int64
```

EDA – Sentiment Distribution

```
In [9]: sns.countplot(data=df,x="sentiment")

plt.title("Sentiment Distribution")

plt.show()
```



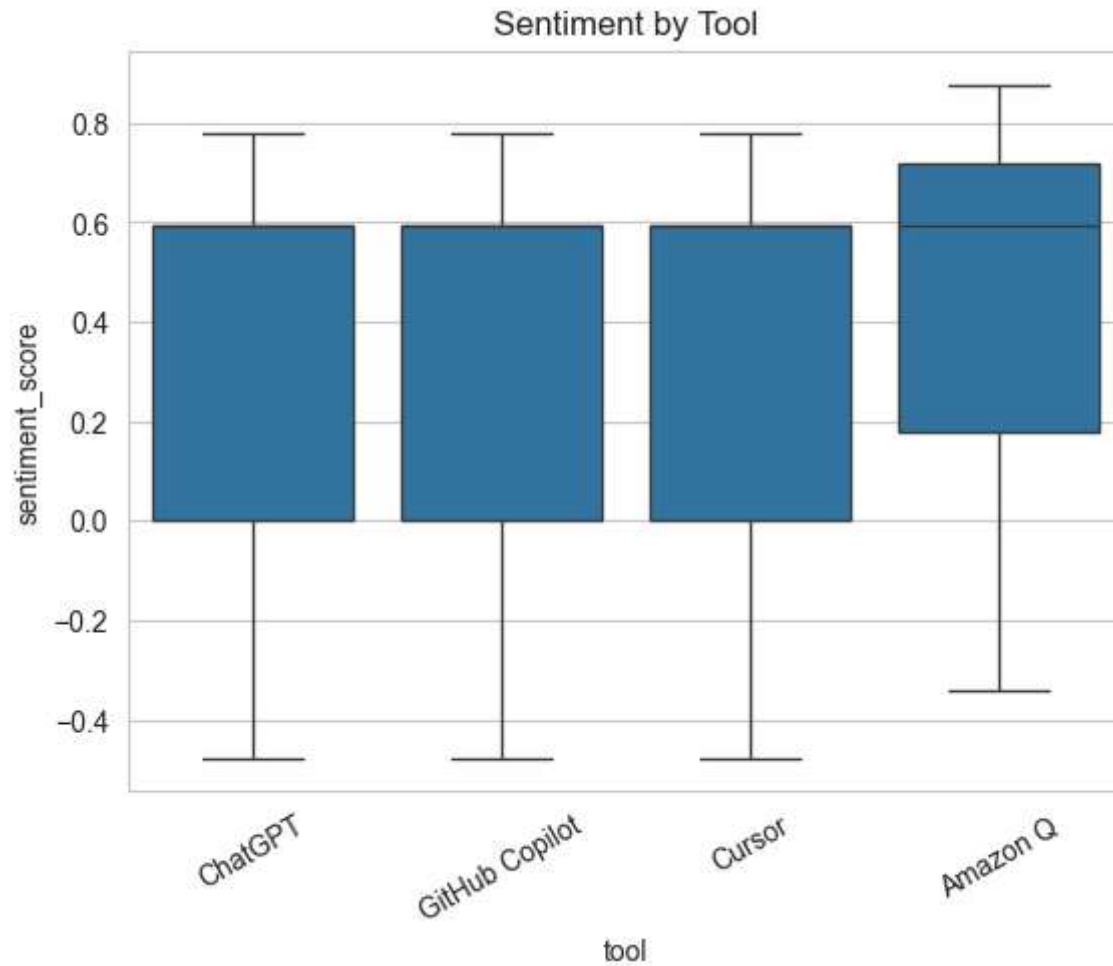
EDA – Tool Comparison

```
In [10]: sns.boxplot(data=df,x="tool",y="sentiment_score")

plt.xticks(rotation=30)

plt.title("Sentiment by Tool")

plt.show()
```

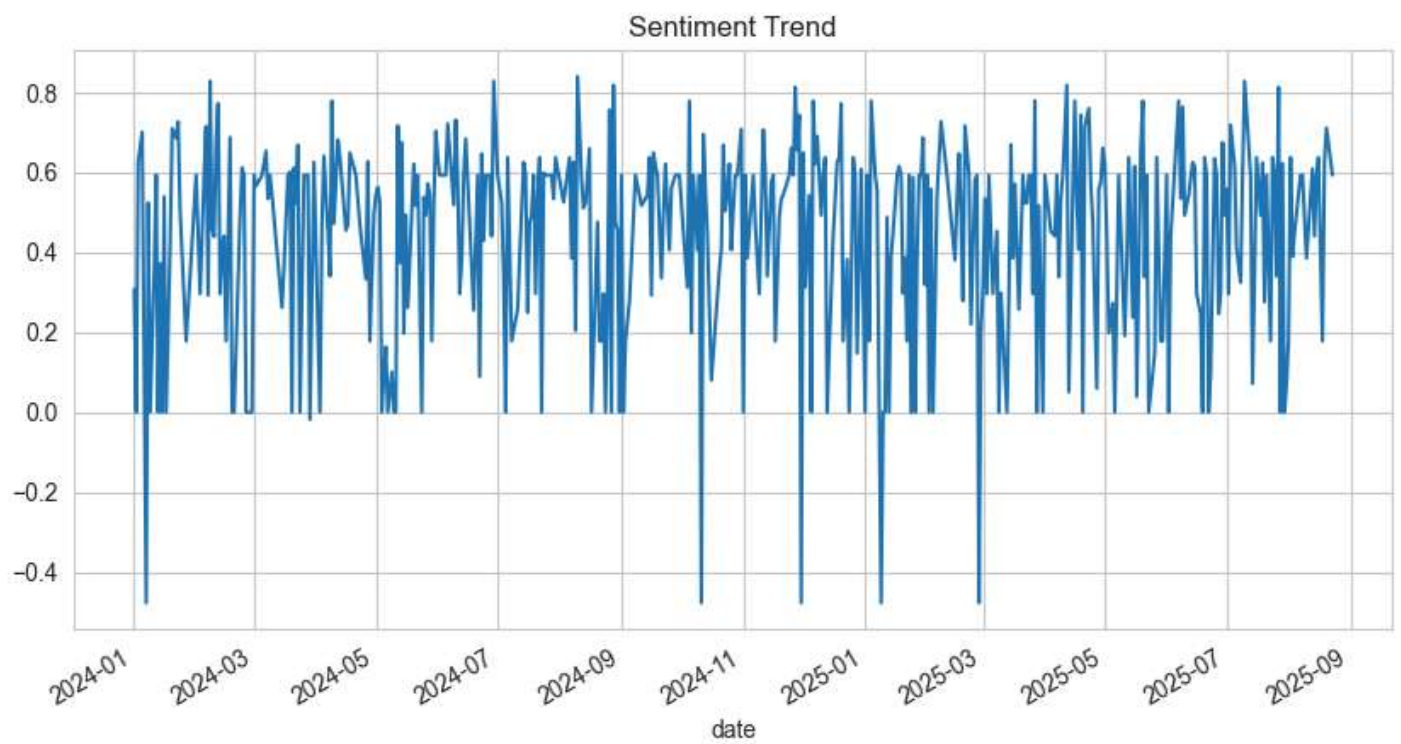


Time Trend Analysis

```
In [11]: df.groupby("date")["sentiment_score"].mean().plot(figsize=(10,5))

plt.title("Sentiment Trend")

plt.show()
```



Feature Sentiment Heatmap

```
In [12]: if len(feature_df)>0:

    pivot=feature_df.groupby(

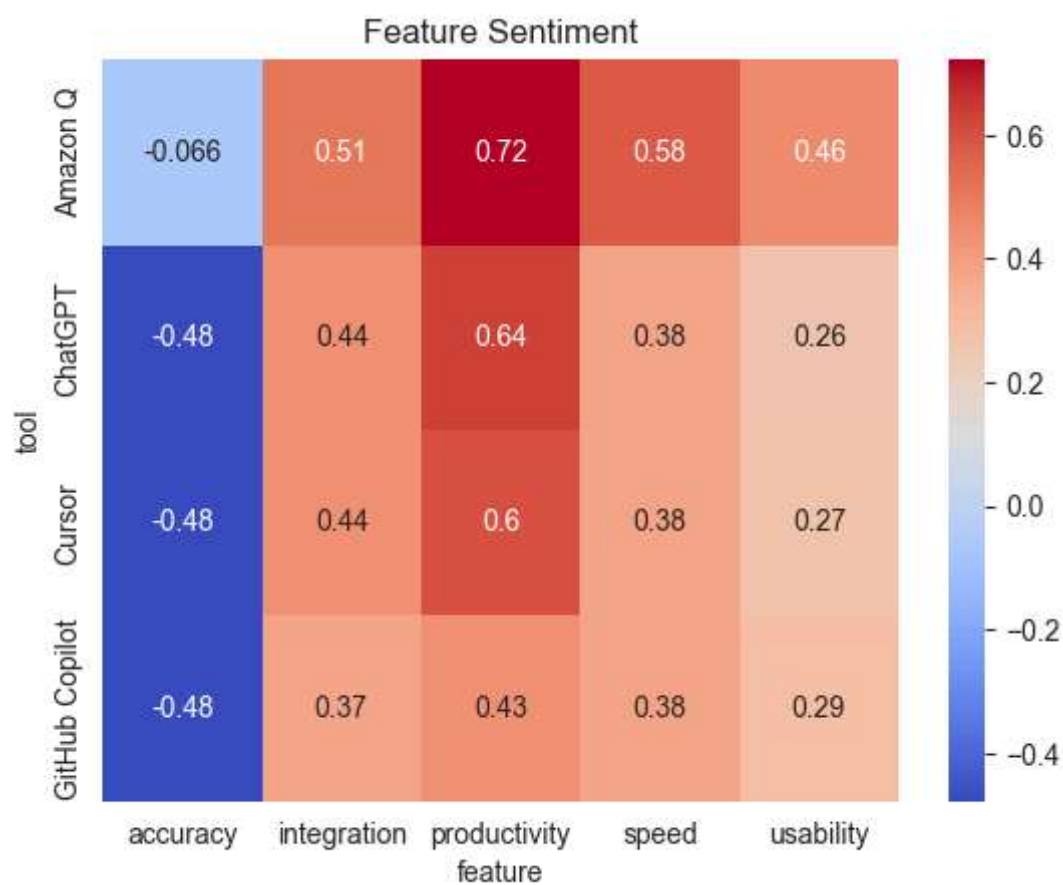
        ["tool","feature"]

    )["sentiment"].mean().unstack()

    sns.heatmap(pivot,annot=True,cmap="coolwarm")

    plt.title("Feature Sentiment")

    plt.show()
```



Topic Extraction

```
In [13]: #####
# Topic Extraction - Word Frequency
#####

vectorizer = CountVectorizer(

    stop_words="english",

    max_features=100

)

X = vectorizer.fit_transform(df["clean"])

word_freq = pd.DataFrame({

    "word":vectorizer.get_feature_names_out(),

    "frequency":X.sum(axis=0).A1

}).sort_values(

    "frequency",

    ascending=False

)

word_freq.head(20)
```

Out[13]:

	word	frequency
26	especially	800
29	experience	320
68	useful	300
53	overall	267
49	mixed	267
17	cursor	200
1	amazon	200
34	github	200
16	copilot	200
9	chatgpt	200
45	integration	120
47	learning	100
54	productivity	94
19	debugging	80
63	suggestions	78
20	development	73
11	code	70
69	workflow	64
37	great	60
56	refactoring	59

```
In [14]: plt.figure(figsize=(10,5))

sns.barplot(

data=word_freq.head(15),

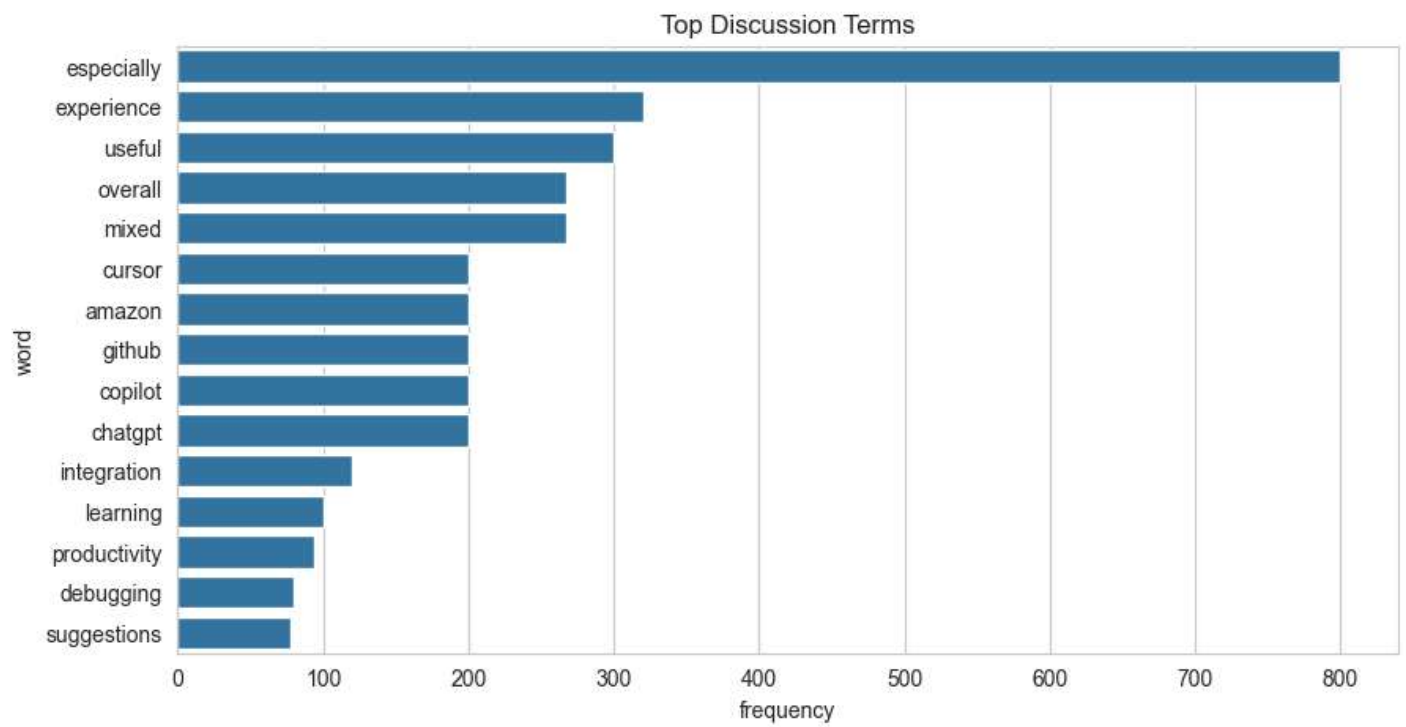
x="frequency",

y="word"

)

plt.title("Top Discussion Terms")

plt.show()
```



```
In [15]: from wordcloud import WordCloud

#####
# Word Cloud Visualization
#####

text = " ".join(df["clean"])

wordcloud = WordCloud(

    width=1000,

    height=500,

    background_color="white",

    colormap="viridis"

).generate(text)

plt.figure(figsize=(12,6))

plt.imshow(wordcloud)

plt.axis("off")

plt.title("AI Tool Discussion Word Cloud")

plt.show()
```

```
for tool in df["tool"].unique():

    temp = df[df["tool"]==tool]

    text = " ".join(temp["clean"])

    wordcloud = WordCloud(

        width=800,

        height=400,

        background_color="white"

    ).generate(text)

    plt.figure(figsize=(10,4))

    plt.imshow(wordcloud)

    plt.axis("off")

    plt.title(tool+" Word Cloud")

    plt.show()
```

[illegible][illegible]

Tool Comparison Dashboard

```
In [17]: dashboard=df.groupby("tool").agg({

    "sentiment_score":"mean",

    "text":"count"

}).rename(columns={"text":"post_volume"}).reset_index()

dashboard
```

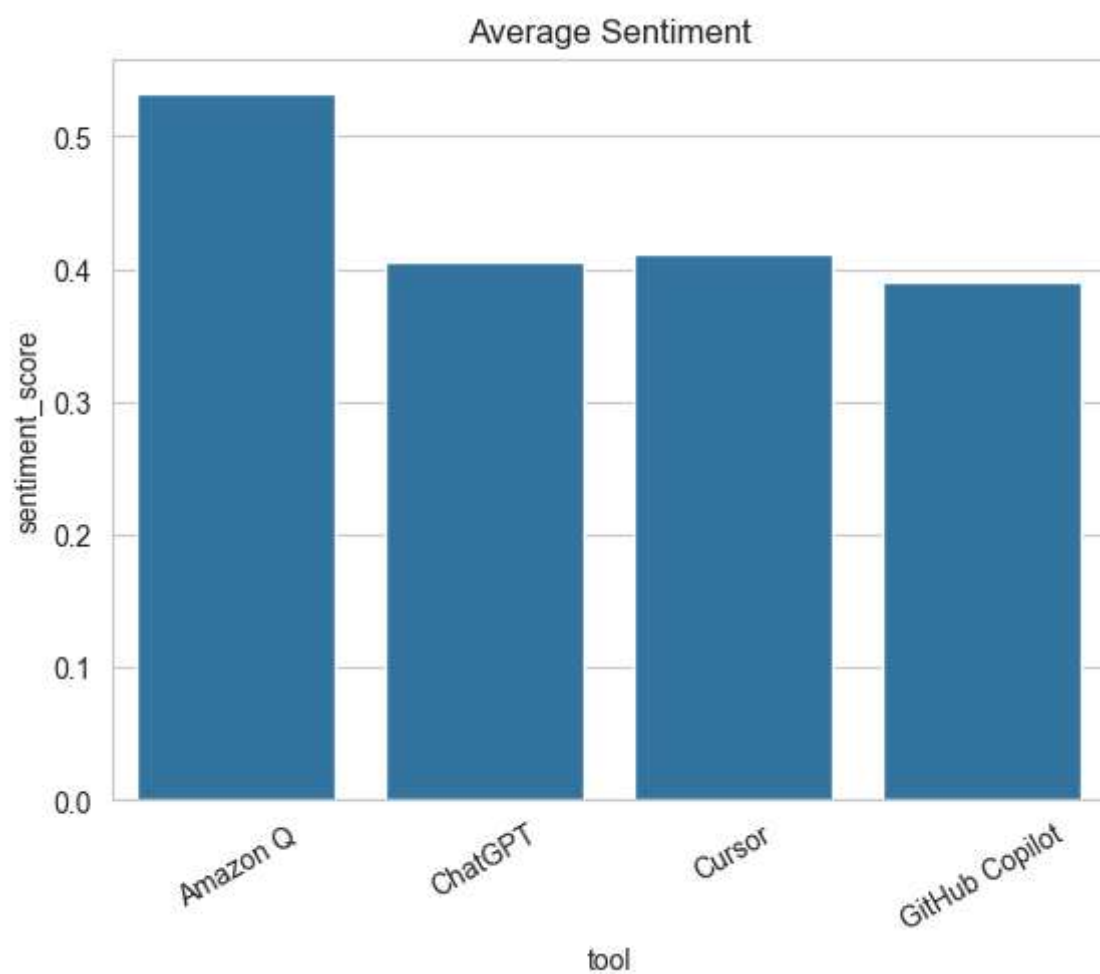
Out[17]:	tool	sentiment_score	post_volume
0	Amazon Q	0.531949	200
1	ChatGPT	0.405844	200
2	Cursor	0.411469	200
3	GitHub Copilot	0.390298	200

```
In [18]: sns.barplot(data=dashboard,x="tool",y="sentiment_score")

plt.title("Average Sentiment")

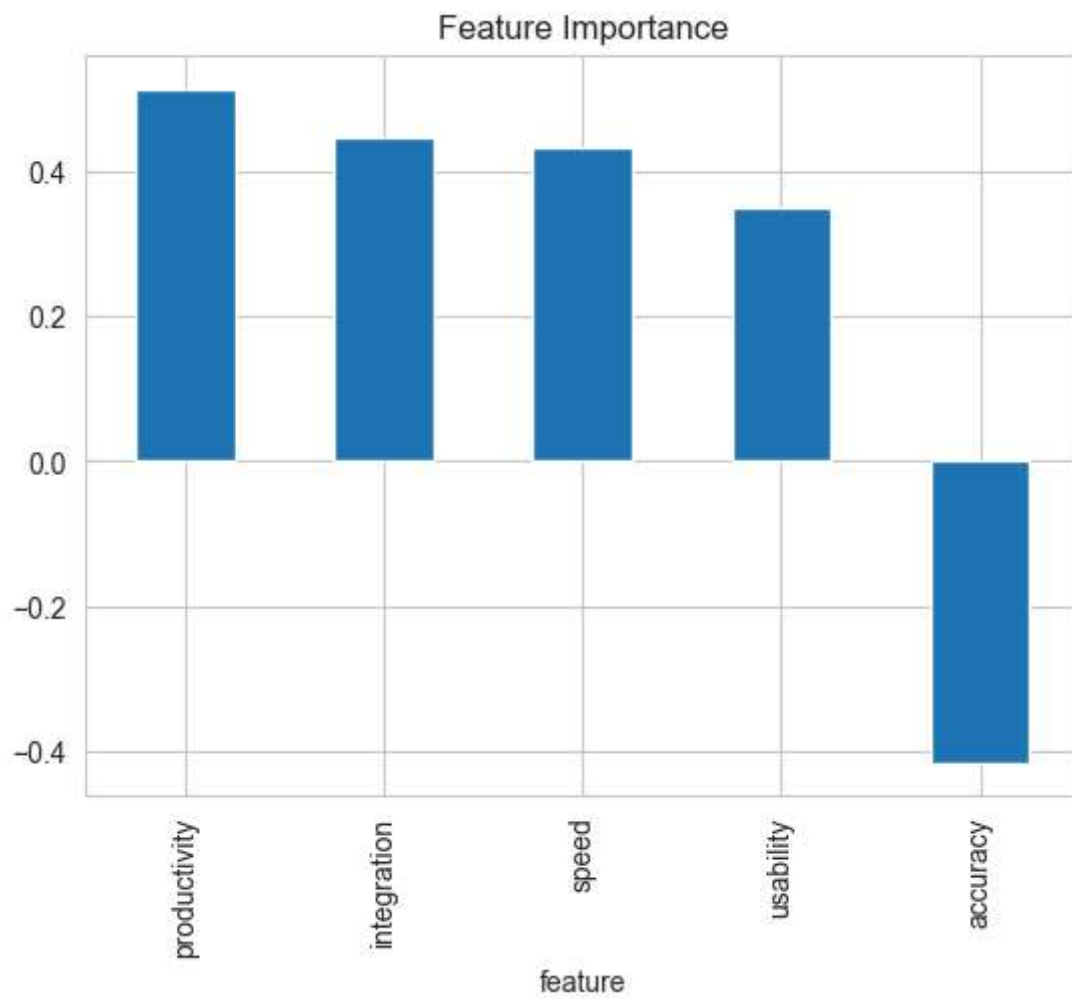
plt.xticks(rotation=30)

plt.show()
```



Feature Importance Ranking

```
In [19]: if len(feature_df)>0:
        feature_rank=feature_df.groupby("feature")["sentiment"].mean().sort_values(ascending=False)
        feature_rank.plot(kind="bar")
        plt.title("Feature Importance")
        plt.show()
```



Executive Insights

```
In [20]: avg=df.groupby("tool")["sentiment_score"].mean()

print("Best sentiment:",avg.idxmax())

print("Lowest sentiment:",avg.idxmin())

print("Most discussed:",dashboard.loc[dashboard["post_volume"].idxmax()]["tool"])
```

Best sentiment: Amazon Q
Lowest sentiment: GitHub Copilot
Most discussed: Amazon Q